UNIVERSITY OF WOLVERHAMPTON

BIRATNAGAR INTERNATIONAL COLLEGE

ing

4CS017

Internet Software Architecture & Database

Final Report on Weather App

Name: Sagar Mishra

ID: 2606063

Module Leader _____

Tutor _____

Table of Contents

Short Reflective Report on Weather webapp final architecture **3**

Screenshot of your data stored in the browser **4**

UML Deployment Diagram **5**

UML Sequence Diagram **6**

UML Activity Diagram **7**

Short Reflective Report on Weather webapp final architecture

The Liteleaf Weather Web App is built using a simple client server setup. On the frontend it is a single-page app made with HTML CSS and JavaScript. This part handles everything the user sees and interacts with like displaying the weather getting your location and saving recently checked weather info in the browser using localStorage. The backend is a small PHP file called connection.php that talks to the OpenWeather API gets the weather data and sends it back to the frontend in JSON format.

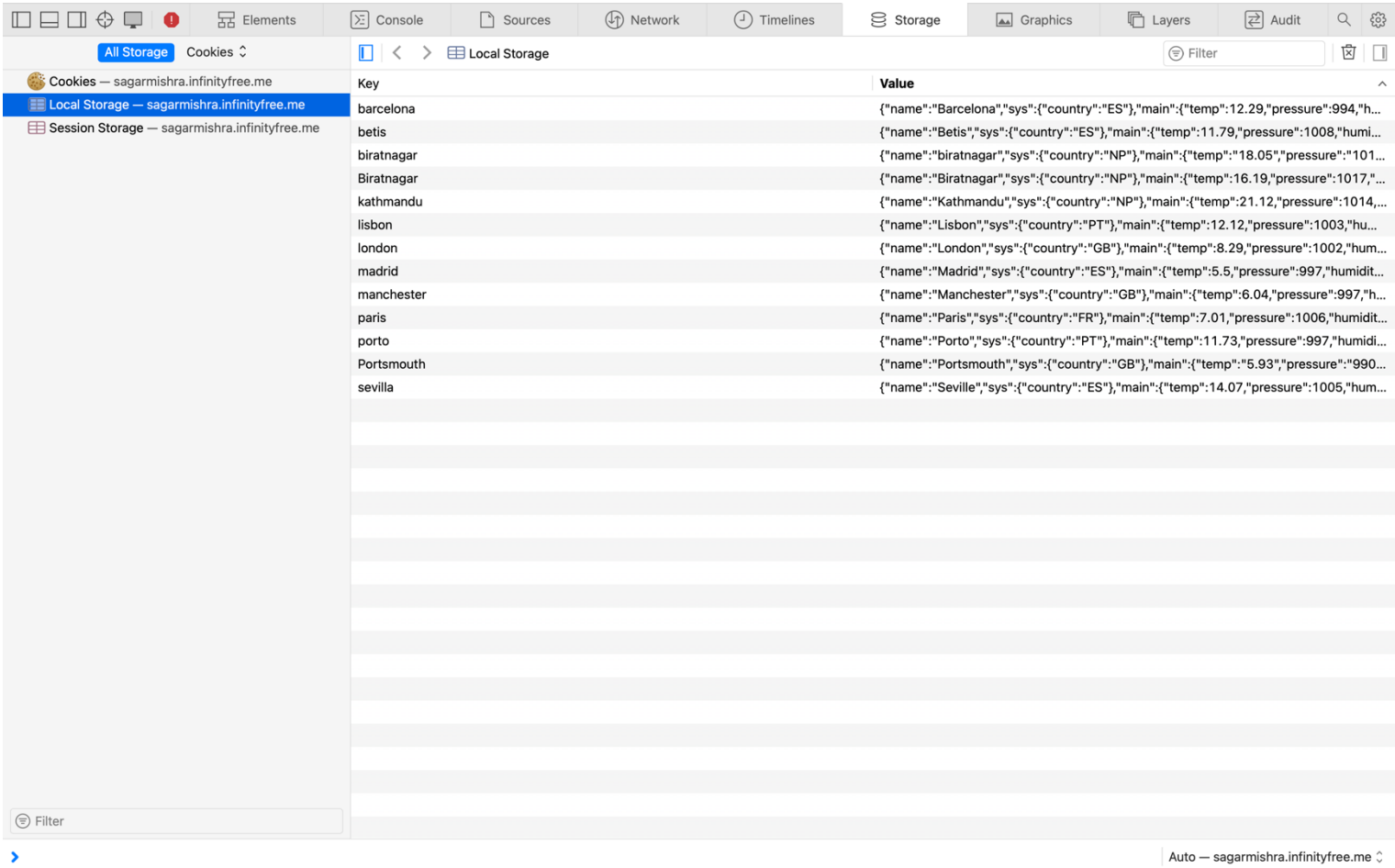
One thing I like about this setup is that each part has its own job. The frontend is all about how the app looks and works for the user while the backend just deals with getting the data from OpenWeather. This makes it easier to fix problems and manage the code. Using localStorage is handy because the app does not have to ask the API for the same info over and over and it can even show weather data when the internet is slow or temporarily offline. Since it is a single-page app everything feels smooth and fast and geolocation makes it convenient by showing the weather for wherever you are automatically.

Of course it is not perfect. Right now the backend has only one PHP endpoint so if many people use the app at once it might get slow. The error messages are basic so users might not know exactly what went wrong. Security could also be better by checking the data more carefully or limiting how many requests the app can make. The localStorage caching is simple too so if someone stays offline for a long time the weather info could get outdated.

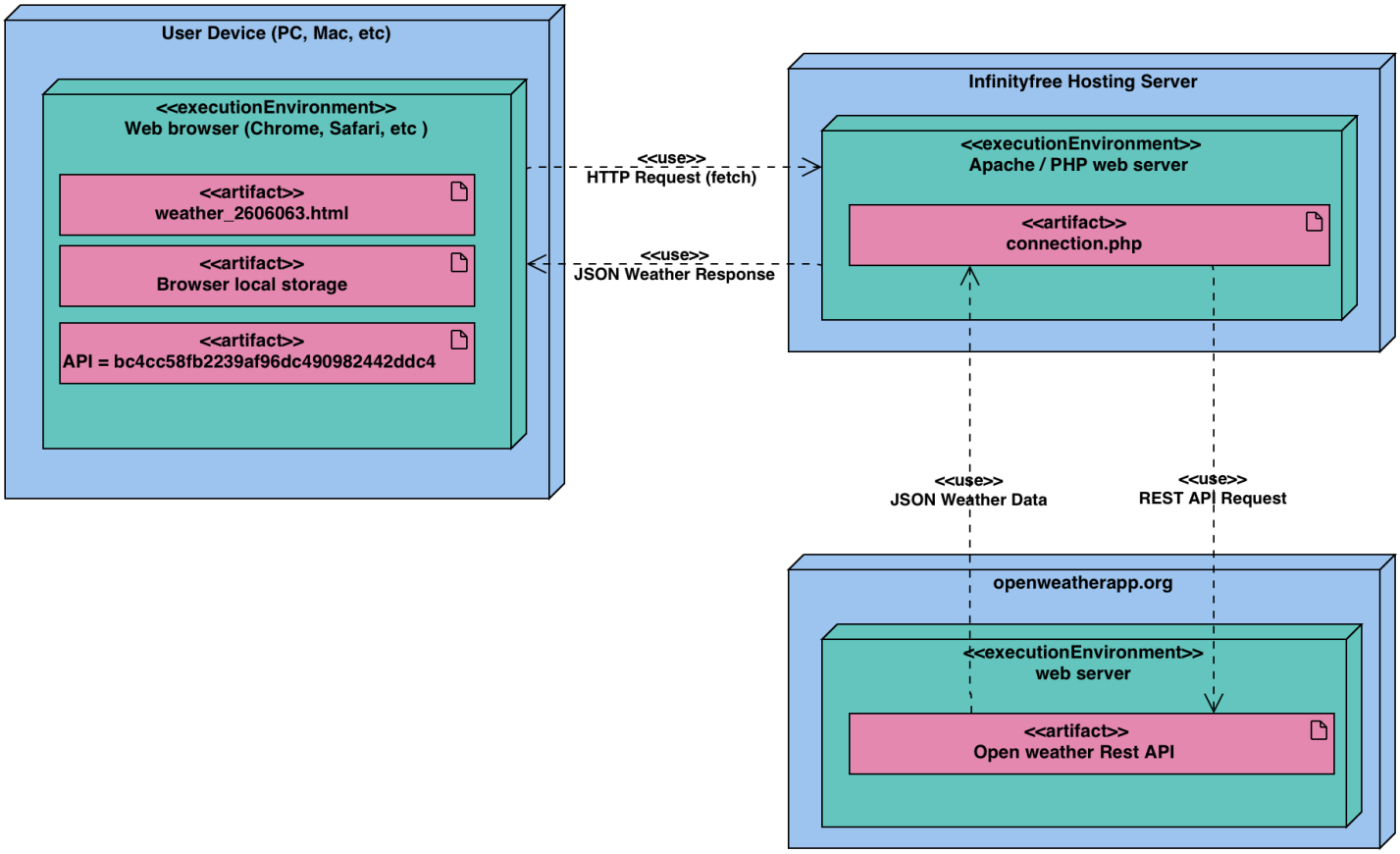
Overall this setup works well for a small to medium web app. It is simple fast and easy to use and there is plenty of room to improve in the future like adding multiple API endpoints user accounts or smarter caching

🔗 **Link of my working app:** https://sagarmishra.infinityfree.me

Screenshot of your data stored in the browser



UML Deployment Diagram



Nodes

- User Device
- InfinityFree Hosting Server
- OpenWeather API Server

Execution Environments

- Web Browser (Safari / Chrome)
- Apache / PHP Web Server
- Web Server

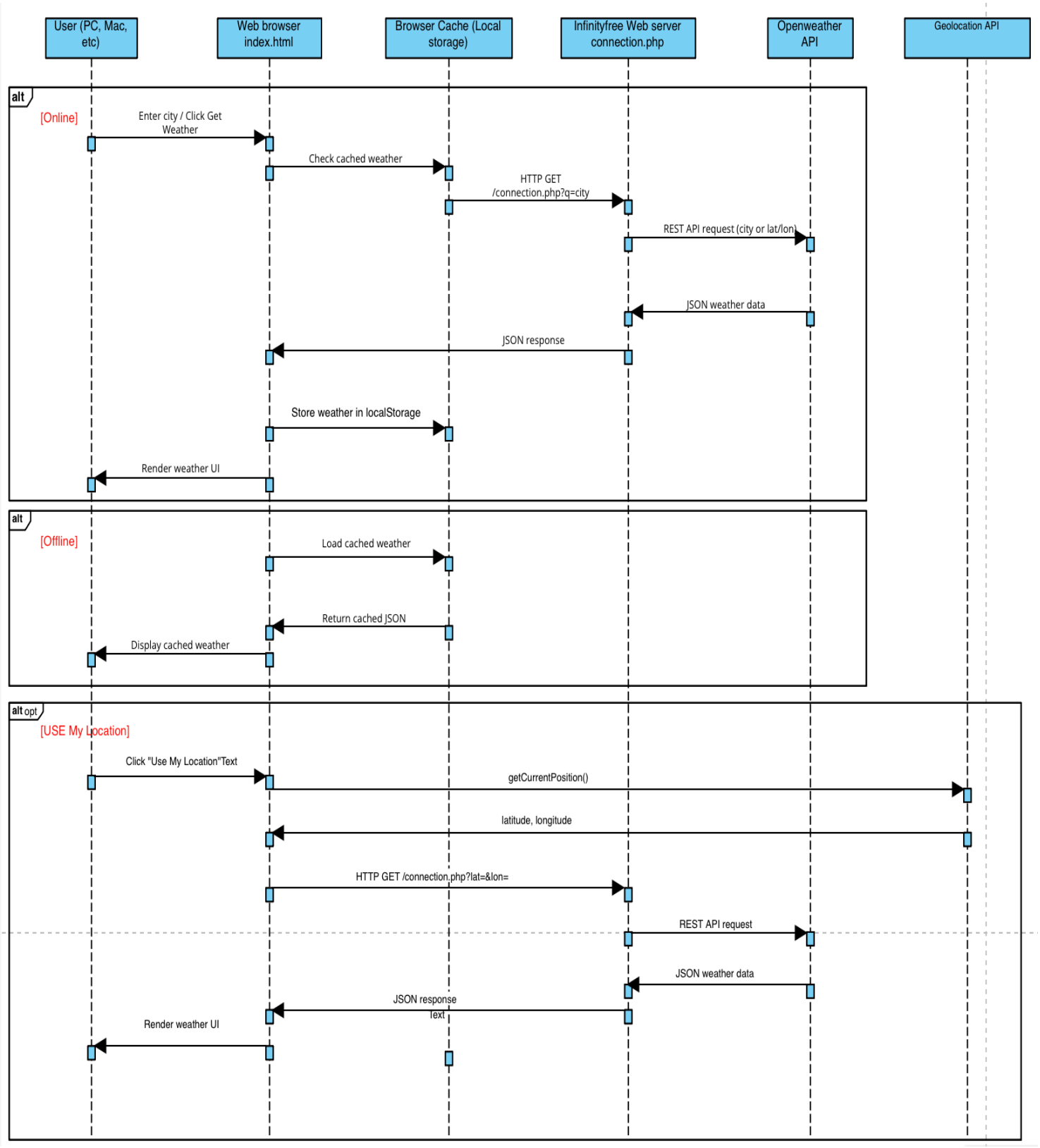
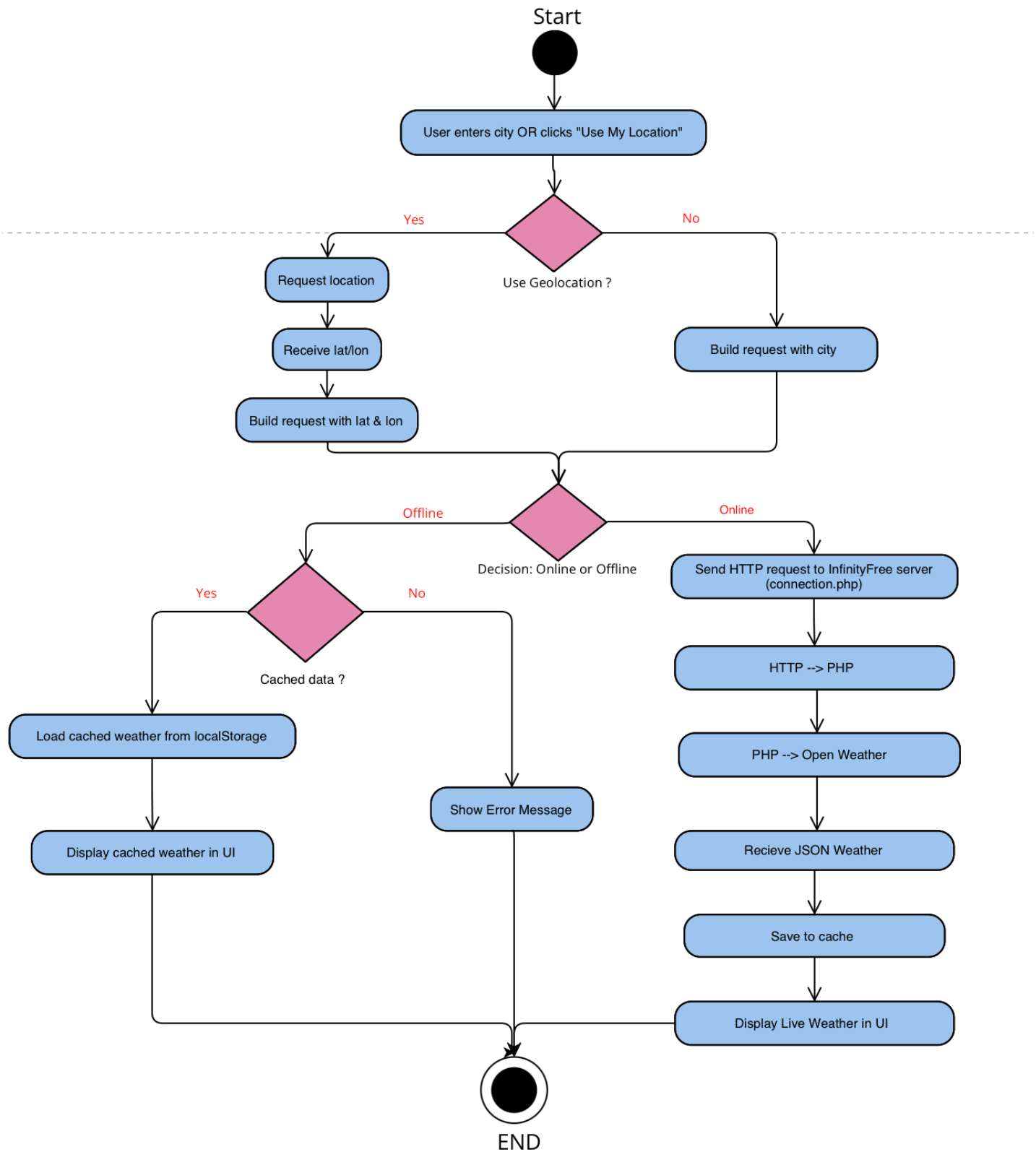
Artifacts

- weather_2606063.html
- connection_2606063.php
- Browser Cache (localStorage)
- Geolocation API
- OpenWeather REST API

Connections

- <<uses>> HTTP Request (fetch)
- <<uses>> JSON Weather Response
- <<uses>> REST API Request
- <<uses>> JSON Weather Data

UML Sequence Diagram



UML Activity Diagram